

PYTHON PROGRAMMING

오프라인

2026.06.29 MON

파이썬 프로그래밍 6일차 강의자료

객체지향 심화 · pandas · matplotlib · API

오늘의 한 줄 목표

상속으로 클래스를 확장하고, 실전 라이브러리를 직접 써본다.

대상: 비전공자 1~4학년 · 구성: 이론 + 실습



오늘의 시간표

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

교시	주제	이론	실습
1	상속 기초	상속·super	Animal→Dog/Cat
2	오버라이딩·캡슐화	private·매직 메서드	도서관·Money
3	외부 라이브러리 & pandas	pip·venv·DF	성적 분석
4	matplotlib 시각화	bar·plot·한글폰트	그래프 저장
5	웹·엑셀 자동화	API·openpyxl	환율·리포트

오늘의 전환

- 상속으로 클래스 재사용
- pandas/matplotlib로 분석·그래프
- requests/openpyxl로 웹·엑셀 자동화

TAKEAWAY

오늘의 목표: 객체지향을 확장하고 실전 라이브러리로 프로젝트 준비를 끝낸다.

복습

지난 시간 복습 & 오늘의 연결고리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

지난 시간까지

- 파일·예외·클래스로 진짜 프로그램의 기본 구조를 완성했습니다.
- Student, Car, BankAccount처럼 데이터와 기능을 한 객체로...

오늘의 해결

- 상속: 비슷한 클래스를 재사용·확장
- pandas·matplotlib: 데이터 분석과 시각화
- requests·openpyxl: 웹 데이터와 엑셀 자동화

끝나면 만들 수 있는 것

TAKEAWAY

오늘은 미니 프로젝트 직전, 객체지향과 실전 도구를 모두 장착하는 날입니다.

1교시

1교시 — 상속(inheritance) 기초

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 기존 클래스를 물려받아 새 클래스를 만든다.

이론 20분

실습 30분

휴식 10분

어제 클래스의 한계

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- eat() 이 똑같이 중복 → 비효율
- 해결: 공통 부분을 물려주기 = 상속!

Python example

```
class Dog:
    def __init__(self, name): self.name = name
    def eat(self): print(f"{self.name} 먹는다")
    def bark(self): print("멍멍")

class Cat:
    def __init__(self, name): self.name = name
    def eat(self): print(f"{self.name} 먹는다")
    # 똑같은 코드 반복!
    def meow(self): print("야옹")
```

TAKEAWAY

eat() 이 똑같이 중복 → 비효율

상속이란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 상속 = 기존 클래스의 속성·메서드를 물려받는 것
- 비유: 부모에게서 유전자를 물려받듯

적용 질문

- 공통 기능은 부모 클래스에 한 번만!
- 자식 클래스는 그걸 그대로 쓰고 + 추가

TAKEAWAY

상속 = 기존 클래스의 속성·메서드를 물려받는 것

상속의 필요성

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 동물원 프로그램: Dog, Cat, Lion, Tiger...
- 모두 공통: 이름, 먹기, 자기
- 각자 다름: 짖기, 야옹, 포효

적용 질문

- 공통은 Animal 부모에, 차이만 자식에!
- → 코드 중복 ↓, 수정 편함 ↑

TAKEAWAY

동물원 프로그램: Dog, Cat, Lion, Tiger...

이론

상속 전 vs 후 (코드 비교)

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

Python example

```
# 상속 전 — eat()가 중복!  
class Dog:  
    def eat(self): print("먹는다")  
    def bark(self): print("멍멍")  
class Cat:  
    def eat(self): print("먹는다") # 또 중복  
    def meow(self): print("야옹")  
# 상속 후 — eat()는 한 번만!  
class Animal:  
    # ... 계속
```

확인 포인트

- 공통 코드가 한 곳에 → 수정도 한 번에

TAKEAWAY

공통 코드가 한 곳에 → 수정도 한 번에

이론

부모 클래스 / 자식 클래스

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 부모 클래스(슈퍼클래스): 물려주는 쪽 (Animal)
- 자식 클래스(서브클래스): 물려받는 쪽 (Dog, Cat)

적용 질문

- 자식은 부모의 모든 것을 자동으로 가짐
- 부모 = 기반, 자식 = 확장

TAKEAWAY

부모 클래스(슈퍼클래스): 물려주는 쪽 (Animal)

상속 문법

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- class 자식(부모): 형태
- 괄호 안에 물려받을 부모 지정

Python example

```
class Animal:      # 부모
    def eat(self):
        print("먹는다")

class Dog(Animal):  # 자식 (괄호에 부모!)
    def bark(self):
        print("멍멍")
```

TAKEAWAY

class 자식(부모): 형태

자식이 부모 기능 사용

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- Dog에 eat·__init__ 안 썼는데 사용 가능!

Python example

```
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self):
        print(f'{self.name} 먹는다')
class Dog(Animal):
    def bark(self):
        print("멍멍")
d = Dog("바둑이")
# ... 계속
```

TAKEAWAY

Dog에 eat·__init__ 안 썼는데 사용 가능!

이론

자식만의 속성·메서드 추가

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 부모 것 + 자식만의 것을 모두 가짐

Python example

```
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name) # 부모 초기화
        self.color = color     # 자식만의 속성
    def meow(self):
        print(f'{self.color} 고양이: 야옹')

c = Cat("나비", "검은")
c.eat() # 부모 기능
c.meow() # 자식 기능
```

TAKEAWAY

부모 것 + 자식만의 것을 모두 가짐

이론

super() — 부모 호출

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `super()` = 부모 클래스를 가리킴
- `super().__init__()` = 부모의 생성자 실행
- 부모가 하던 초기화를 재활용!

Python example

```
class Dog(Animal):  
    def __init__(self, name, breed):  
        super().__init__(name) # 부모의  
            __init__ 호출  
        self.breed = breed
```

TAKEAWAY

`super()` = 부모 클래스를 가리킴

이론

super() 왜 필요한가

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 자식이 `__init__` 을 새로 쓰면 부모 것은 자동 실행 안 됨
- `super()` 로 부모 초기화를 명시적으로 호출해야 함

Python example

```
class Cat(Animal):  
    def __init__(self, name, color):  
        # super().__init__(name) 없으면  
        # self.name 이 안 만들어짐!  
        super().__init__(name) # 부모가 name  
        세팅  
        self.color = color
```

TAKEAWAY

자식이 `__init__` 을 새로 쓰면 부모 것은 자동 실행 안 됨

이론

상속 계층 (여러 단계)

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 상속은 여러 단계로 이어질 수 있음
- 할아버지 → 부모 → 자식
- 깊게는 x, "단계가 이어질 수 있다" 정도

Python example

```
class Animal: ...  
class Pet(Animal): ...    # Animal 상속  
class Dog(Pet): ...      # Pet 상속 (간접적으로  
    Animal도)
```

TAKEAWAY

상속은 여러 단계로 이어질 수 있음

is-a 관계

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 상속은 "~은 ~이다(is-a)" 관계일 때 적합
- "Dog is a Animal" (개는 동물이다) → 상속 OK

적용 질문

- "EBook is a Book" (전자책은 책이다) → 상속 OK
- "Car has a Engine" (가짐) → 상속 아님 (속성으로)

TAKEAWAY

상속은 "~은 ~이다(is-a)" 관계일 때 적합

이론

object — 모든 클래스의 조상

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 모든 클래스는 자동으로 object 를 상속 받음
- 그래서 우리가 안 만들어도 `__str__` 같은 기본 기능을 이미...
- 우리가 `__str__` 을 정의하면? → 기본 것을 덮어쓰는(오버라이딩)셈
- "모든 객체는 사실 이미 무언가를 물려받았다"

Python example

```
class Dog:    # 사실은 class Dog(object):  
    pass
```

TAKEAWAY

모든 클래스는 자동으로 object 를 상속받음

이론

상속을 언제 쓸까?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 여러 클래스에 공통 기능이 많을 때
- "종류"가 나뉘는 개념: 동물→개/고양이, 도형→원/사각형

적용 질문

- 공통은 부모, 특수는 자식
- 억지 상속은 금물 — 진짜 is-a 관계일 때만

TAKEAWAY

여러 클래스에 공통 기능이 많을 때

정리

상속 정리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

오늘의 결과물

- class 자식(부모): 로 물려받기
- 자식은 부모의 속성·메서드를 자동으로 가짐
- `super().__init__()` 으로 부모 초기화 재활용
- 자식만의 속성·메서드 추가 가능

다음 연결

- is-a 관계일 때 사용

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.

실습

Animal → Dog / Cat

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self):
        print(f'{self.name}이(가) 먹습니다')
class Dog(Animal):
    def sound(self):
        # ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

super()로 속성 확장

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age
class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age) # 부모 초기화
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

상속 설계 (택1 이상)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

따라 하기

- Vehicle → Car / Bike: 공통 move(), 각자 다른 속성
- Person → Student / Teacher: 공통 이름, 각자 역할...

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- Device → Phone / Tablet: 공통 전원, 각자 기능
- Food → Pizza / Burger: 공통 가격, 각자 조리법

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 도구로 넘어갈 준비

10 MIN BREAK

다음 교시: 오버라이딩·캡슐화·매직 메서드 — 객체를 더 똑똑하게!

작성 파일 저장

설치 상태 확인

실행 결과 캡처

프로젝트 메모

TAKEAWAY

라이브러리 실습 전후에는 설치 상태와 생성된 파일을 함께 확인합니다.

2교시

2교시 — 오버라이딩·캡슐화·매직 메서드

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 메서드 재정의·캡슐화·매직 메서드로 클래스를 정교하게 만든다.

이론

메서드 오버라이딩이란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 오버라이딩 = 부모의 메서드를 자식이 다시 정의(덮어쓰기)
- 같은 이름, 다른 동작

Python example

```
class Animal:
    def sound(self):
        print("...")

class Dog(Animal):
    def sound(self): # 부모 것을 덮어씀!
        print("멍멍")
```

TAKEAWAY

오버라이딩 = 부모의 메서드를 자식이 다시 정의(덮어쓰기)

이론

오버라이딩 예시

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

Python example

```
class Animal:
    def sound(self):
        print("동물 소리")
class Dog(Animal):
    def sound(self):
        print("멍멍")
class Cat(Animal):
    def sound(self):
        print("야옹")
# ... 계속
```

확인 포인트

- 같은 sound() 인데 클래스마다 다르게 동작!

TAKEAWAY

같은 sound() 인데 클래스마다 다르게 동작!

실습

오버라이딩 + super() 함께

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Animal:
    def info(self):
        print("나는 동물입니다")
class Dog(Animal):
    def info(self):
        super().info()    # 부모 것도 실행
        print("그리고 개입니다")
# ... 계속
```

실습 포인트

- 부모 기능을 유지하면서 추가할 때 super() 활용

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

이론

다형성(polymorphism)

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 다형성 = 같은 명령이 객체마다 다르게 동작
- 똑같은 sound() 호출인데 객체 종류에 따라 결과가 다름
- 객체지향의 강력한 특징!

Python example

```
animals = [Dog(), Cat(), Animal()]  
for a in animals:  
    a.sound() # 각자 다른 소리!
```

TAKEAWAY

다형성 = 같은 명령이 객체마다 다르게 동작

이론

다형성의 장점

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 함수가 종류를 몰라도 동작 → 유연한 코드
- 새 동물(Lion)을 추가해도 함수 수정 불필요!

Python example

```
def make_sound(animal):  
    animal.sound() # 어떤 동물이든 OK!  
  
make_sound(Dog()) # 멍멍  
make_sound(Cat()) # 야옹
```

TAKEAWAY

함수가 종류를 몰라도 동작 → 유연한 코드

캡슐화란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 캡슐화 = 중요한 데이터를 보호하고 숨기기
- 비유: 약 캡슐이 내용물을 감싸듯

적용 질문

- 직접 건드리면 안 되는 속성을 막아두기
- 예: 은행 잔액을 아무나 직접 바꾸면 안 됨!

TAKEAWAY

캡슐화 = 중요한 데이터를 보호하고 숨기기

이론

private 속성

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 속성 앞 __(밑줄 2개) = 외부에서 직접 접근 차단

Python example

```
class BankAccount:
    def __init__(self):
        self.__balance = 0    # __ = private
                               (외부 차단)
    def deposit(self, amount):
        self.__balance += amount
acc = BankAccount()
acc.deposit(1000)
# acc.__balance = -999 → 직접 접근 막힘!
```

TAKEAWAY

속성 앞 __(밑줄 2개) = 외부에서 직접 접근 차단

이론

캡슐화가 필요한 이유

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 데이터를 검증 없이 막 바꾸는 것 방지
- 정해진 메서드를 통해서만 변경 → 안전

Python example

```
# 캡슐화 없으면
acc.balance = -50000    # 마이너스 잔액?!

# 캡슐화 + 메서드로
acc.withdraw(50000)    # 잔액 검증 후 안전하게
```

TAKEAWAY

데이터를 검증 없이 막 바꾸는 것 방지

이론

getter / setter

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- private 속성을 안전하게 읽고(get) 쓰는(set) 통로

Python example

```
class Person:
    def __init__(self, age):
        self.__age = age
    def get_age(self):      # getter (읽기)
        return self.__age
    def set_age(self, age): # setter (검증 후
                           # 쓰기)
        if age >= 0:
            self.__age = age
# ... 계속
```

TAKEAWAY

private 속성을 안전하게 읽고(get) 쓰는(set) 통로

이론

매직 메서드란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 매직 메서드 = `__이름__` 형태의 특별한 메서드
- 특정 상황에서 자동으로 호출됨

적용 질문

- 이미 본 것: `__init__` (생성 시 자동!)
- 객체를 파이썬 문법과 자연스럽게 연결

TAKEAWAY

매직 메서드 = `__이름__` 형태의 특별한 메서드

이론

__init__ 도 매직 메서드

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- __init__ = 인스턴스 생성 시 자동 호출되는 매직 메서드
- 우리가 이미 매일 쓰고 있었음!

Python example

```
class Dog:
    def __init__(self, name): # 생성 시 자동 호출!
        self.name = name

d = Dog("바둑이") # __init__ 자동 실행
```

TAKEAWAY

__init__ = 인스턴스 생성 시 자동 호출되는 매직 메서드

이론

__str__ — 출력 형태 정하기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- print(객체) 할 때 자동 호출
- 없으면 <__main__.Dog object at 0x...>...

Python example

```
class Dog:
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return f"강아지 {self.name}"

print(Dog("바둑이")) # 강아지 바둑이
```

TAKEAWAY

print(객체) 할 때 자동 호출

이론

__len__ — len() 지원

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- len(객체) 할 때 자동 호출
- 내 객체도 len() 으로 셀 수 있게!

Python example

```
class Team:
    def __init__(self, members):
        self.members = members
    def __len__(self):
        return len(self.members)

team = Team(["철수", "영희", "민수"])
print(len(team))  # 3
```

TAKEAWAY

len(객체) 할 때 자동 호출

이론

__eq__ — == 비교 지원

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- == 비교 시 자동 호출
- 객체끼리 내용으로 비교 가능하게!

Python example

```
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __eq__(self, other):
        return self.amount == other.amount

print(Money(1000) == Money(1000)) # True
```

TAKEAWAY

== 비교 시 자동 호출

정리

매직 메서드 정리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

매직 메서드	호출 시점
<code>__init__</code>	객체 생성 시
<code>__str__</code>	<code>print(객체)</code>
<code>__len__</code>	<code>len(객체)</code>
<code>__eq__</code>	객체 == 객체
<code>__add__</code>	객체 + 객체

제출/마무리

- 다 외울 필요 x — "객체를 파이썬답게 만든다" 정도

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.

실습

도서관 시스템 (상속 + 오버라이딩)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Book:
    def __init__(self, title):
        self.title = title
    def info(self):
        return f"[책] {self.title}"
class EBook(Book):
    def __init__(self, title, size):
        # ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

Money 클래스 (매직 메서드)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __str__(self):
        return f"{self.amount:,}원"
    def __add__(self, other):
        return Money(self.amount +
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

심화 클래스 (택1 이상)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

따라 하기

- AudioBook(Book): 재생시간 속성, info 오버라이딩
- Point 클래스: __add__로 좌표 덧셈, __str__로 (x, y)

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- Cart 클래스: __len__으로 담긴 상품 수
- SecureAccount: private 잔액 + getter/setter...

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 도구로 넘어갈 준비

10 MIN BREAK

다음 교시: 외부 라이브러리 & pandas — 세상이 쓰는 데이터 도구!

작성 파일 저장

설치 상태 확인

실행 결과 캡처

프로젝트 메모

TAKEAWAY

라이브러리 실습 전후에는 설치 상태와 생성된 파일을 함께 확인합니다.

3교시

3교시 — 외부 라이브러리 & pandas

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: pip로 외부 도구를 설치하고, pandas로 데이터를 분석한다.

이론 20분

실습 30분

휴식 10분

이론

표준 vs 외부 라이브러리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 표준 라이브러리: 파이썬에 기본 포함 (math, random, json...)...
- 외부 라이브러리: 따로 설치(pip) 해야 함 (pandas, matplotlib...

적용 질문

- 외부엔 전 세계 개발자가 만든 강력한 도구가 가득!

TAKEAWAY

표준 라이브러리: 파이썬에 기본 포함 (math, random, json...) → 바로 import

이론

라이브러리 생태계

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 파이썬 패키지 저장소(PyPI)에 50만 개 이상의 라이브러리!
- 데이터·AI·웹·게임·자동화... 거의 모든 분야

적용 질문

- 비전공자도 가져다 쓰기만 하면 전문가 기능 사용
- "직접 만들기 전에 라이브러리부터 검색!"

TAKEAWAY

파이썬 패키지 저장소(PyPI)에 50만 개 이상의 라이브러리!

이론

pip — 설치 도구

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- pip install 이름 으로 설치
- 터미널(VS Code 터미널)에서 실행
- 한 번 설치하면 계속 사용 가능

Python example

```
pip install pandas      # 터미널에서  
pip install matplotlib  
pip install requests openpyxl
```

TAKEAWAY

pip install 이름 으로 설치

이론

가상환경(venv) 왜 필요?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 프로젝트마다 쓰는 라이브러리·버전이 다름
- 한 컴퓨터에 다 섞이면 충돌 발생

적용 질문

- 가상환경 = 프로젝트별 독립된 작업 공간
- 비유: 프로젝트마다 분리된 서랍

TAKEAWAY

프로젝트마다 쓰는 라이브러리·버전이 다름

이론

venv 만들기 (참고)

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 활성화하면 그 안에서만 라이브러리 설치
- 오늘은 없이 진행해도 OK, "이런 게 있다"만 알기

Python example

```
python -m venv myenv    # 가상환경 생성
# 활성화
myenv\Scripts\activate  # Windows
source myenv/bin/activate # Mac
```

TAKEAWAY

활성화하면 그 안에서만 라이브러리 설치

이론

import as 관례

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 자주 쓰는 라이브러리는 짧은 별칭(3일차 as 복습)
- pd, plt, np 는 전 세계 공통 관례
- 코드를 짧고 읽기 쉽게

Python example

```
import pandas as pd      # pandas를 pd로  
import matplotlib.pyplot as plt
```

TAKEAWAY

자주 쓰는 라이브러리는 짧은 별칭(3일차 as 복습)

이론

분야별 대표 라이브러리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

분야	라이브러리	용도
데이터	pandas	표 데이터 분석
시각화	matplotlib	그래프
웹	requests	API 호출
크롤링	BeautifulSoup	웹페이지 추출
엑셀	openpyxl	엑셀 자동화

읽는 법

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

pandas란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- pandas = 파이썬으로 엑셀 같은 데이터를 다루는 도구
- 수만~수십만 줄 데이터도 순식간에 처리

적용 질문

- CSV·엑셀 읽기, 통계, 정렬, 필터링...
- 데이터 분석의 국민 라이브러리

TAKEAWAY

pandas = 파이썬으로 엑셀 같은 데이터를 다루는 도구

이론

DataFrame — 표 자료구조

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- DataFrame = 엑셀의 표 (행 + 열)
- 딕셔너리로 만들 수도, CSV에서 읽을 수도
- pandas의 핵심 자료구조

Python example

```
import pandas as pd
df = pd.DataFrame({
    "이름": ["홍길동", "김철수"],
    "점수": [90, 85]
})
print(df)
```

TAKEAWAY

DataFrame = 엑셀의 표 (행 + 열)

이론

CSV 읽기 — read_csv

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `pd.read_csv("파일")` 한 줄로 CSV를 표로 !
- 어제 `csv` 모듈로 한 줄씩 읽던 것이 한 줄로
- 엑셀 파일은 `read_excel`

Python example

```
import pandas as pd
df = pd.read_csv("scores.csv")
print(df)
```

TAKEAWAY

`pd.read_csv("파일")` 한 줄로 CSV를 표로!

이론

데이터 미리보기 — head

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 큰 데이터의 일부만 빠르게 확인
- 데이터 분석의 첫 단계 = 살펴보기

Python example

```
df.head()      # 처음 5줄  
df.head(3)     # 처음 3줄  
df.tail()      # 마지막 5줄  
df.shape       # (행 수, 열 수)  
df.columns     # 열 이름들
```

TAKEAWAY

큰 데이터의 일부만 빠르게 확인

이론

열 선택 & 통계

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 열을 고르고 통계 함수 한 방에
- 어제 직접 반복문 짜던 게 메서드 하나로!

Python example

```
df["점수"]          # 점수 열만  
df["점수"].mean()   # 평균  
df["점수"].max()    # 최고  
df["점수"].sum()    # 합계  
df["점수"].describe() # 통계 요약 한 번에!
```

TAKEAWAY

열을 고르고 통계 함수 한 방에

이론

정렬 & 필터링

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `sort_values` 로 정렬, 조건으로 필터링
- 수백 줄도 한 줄로 정렬·선별

Python example

```
df.sort_values("점수", ascending=False) # 점수  
내림차순  
df[df["점수"] >= 90] # 90점  
이상만
```

TAKEAWAY

`sort_values` 로 정렬, 조건으로 필터링

이론

새 열 추가하기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `df["새열"] = ...` 로 열을 추가
- 기존 열을 계산해 새 열 만들기도 가능
- 엑셀에서 새 칸 만들어 수식 넣는 것과 같음

Python example

```
df["등급"] = ["A", "B", "A", "C"] # 직접 값 넣기  
df["보너스점수"] = df["점수"] + 5 # 계산으로 새 열!  
print(df)
```

TAKEAWAY

`df["새열"] = ...` 로 열을 추가

정리

pandas 정리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

오늘의 결과물

- `pd.read_csv()` 로 데이터 불러오기
- `head()` `shape` `columns` 로 살펴보기
- `df["열"].mean()` 등 통계
- `sort_values()` 정렬, `df[조건]` 필터링

다음 연결

- 데이터 분석의 80%는 이걸로 해결!

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.

실습

성적 데이터 분석

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import pandas as pd
# 데이터 만들기
df = pd.DataFrame({
    "이름": ["홍길동", "김철수", "이영희", "박민수"],
    "점수": [90, 85, 95, 70]
})
print(df)
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

pandas 분석 (택1 이상)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

따라 하기

- 상품 데이터: 상품·가격 DataFrame → 평균가, 최고가 상품
- 반별 성적: 이름·반·점수 → 90점 이상만 필터링

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- 판매 데이터: 월·매출 → 정렬해서 최고 매출 달 찾기
- 어제 만든 scores.csv 를 read_csv 로 읽어 분석

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 도구로 넘어갈 준비

10 MIN BREAK

다음 교시: matplotlib — 데이터를 그래프로 그리기
!

작성 파일 저장

설치 상태 확인

실행 결과 캡처

프로젝트 메모

TAKEAWAY

라이브러리 실습 전후에는 설치 상태와 생성된 파일을 함께 확인합니다.

4교시

4교시 — matplotlib 시각화

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: matplotlib으로 데이터를 그래프로 시각화한다.

이론 20분

실습 30분

휴식 10분

이론

왜 시각화인가?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 숫자만 나열하면 한눈에 안 들어옴
- 그래프로 그리면 차이·추세가 즉시 보임
- "백 마디 숫자보다 한 장의 그래프"

Python example

홍길동 90, 김철수 85, 이영희 95, 박민수 70, ...

TAKEAWAY

숫자만 나열하면 한눈에 안 들어옴

이론

matplotlib란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- matplotlib = 파이썬 대표 그래프 라이브러리
- 막대·선·원·산점도 등 거의 모든 그래프

적용 질문

- pandas와 짝꿍 (데이터 → 그래프)
- 보고서·논문·대시보드의 단골

TAKEAWAY

matplotlib = 파이썬 대표 그래프 라이브러리

기본 구조

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- plt.그래프종류(...) → 그리고
- plt.show() → 띄우기
- 이 두 단계가 기본!

Python example

```
import matplotlib.pyplot as plt

plt.bar(["A", "B", "C"], [3, 7, 5]) # 그래프
                                  # 그리기
plt.show()                        # 화면에
                                  # 보여주기
```

TAKEAWAY

plt.그래프종류(...) → 그리고

이론

막대 그래프 — bar

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `plt.bar(x축, y축)` → 막대 그래프
- 항목별 비교에 최적

Python example

```
import matplotlib.pyplot as plt

names = ["국어", "영어", "수학"]
scores = [90, 85, 95]

plt.bar(names, scores)
plt.show()
```

TAKEAWAY

`plt.bar(x축, y축)` → 막대 그래프

이론

선 그래프 — plot

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- plt.plot() → 선 그래프
- 시간에 따른 변화. 추세에 최적

Python example

```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5]
sales = [100, 120, 90, 150, 200]

plt.plot(months, sales)
plt.show()
```

TAKEAWAY

plt.plot() → 선 그래프

이론

제목과 축 이름

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 제목·축 라벨이 있어야 이해되는 그래프
- 항상 붙이는 습관!

Python example

```
plt.bar(names, scores)
plt.title("과목별 점수")    # 제목
plt.xlabel("과목")         # x축 이름
plt.ylabel("점수")         # y축 이름
plt.show()
```

TAKEAWAY

제목·축 라벨이 있어야 이해되는 그래프

이론

한글 깨짐 해결

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 기본 폰트는 한글 미지원 → 네모(□) 로 깨짐
- 폰트 설정 한 줄로 해결!

Python example

```
import matplotlib.pyplot as plt
plt.rcParams["font.family"] = "Malgun Gothic"
# Windows
# plt.rcParams["font.family"] = "AppleGothic"
# Mac
plt.rcParams["axes.unicode_minus"] = False
# 마이너스 깨짐 방지
```

TAKEAWAY

기본 폰트는 한글 미지원 → 네모(□) 로 깨짐

이론

여러 막대 색 지정

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- color 로 막대 색 지정 (색 이름 또는 리스트)
- 강조하고 싶은 항목만 다른 색도 가능

Python example

```
plt.bar(names, scores, color=["red", "green",  
                             "blue"])  
plt.bar(names, scores, color="skyblue") # 전체  
한 색
```

TAKEAWAY

color 로 막대 색 지정 (색 이름 또는 리스트)

이론

파이 차트

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- plt.pie() → 원형 비율 그래프
- autopct 로 퍼센트 표시
- 비율·점유율 표현에 최적

Python example

```
plt.pie([30, 50, 20], labels=["A", "B", "C"],  
        autopct="%1.1f%%")  
plt.show()
```

TAKEAWAY

plt.pie() → 원형 비율 그래프

이론

여러 데이터 + 범례

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 한 그래프에 여러 데이터를 겹쳐 비교
- label + legend() 로 무엇이 무엇인지 표시
- plt.figure(figsize=(10, 5)) 로 그래프 크...

Python example

```
plt.plot(months, sales_a, label="A상품")
plt.plot(months, sales_b, label="B상품")
plt.legend()      # 범례(어느 선이 뭔지) 표시
plt.show()
```

TAKEAWAY

한 그래프에 여러 데이터를 겹쳐 비교

이론

그래프 저장

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- `savefig("파일.png")` 로 이미지 저장
- 보고서·발표 자료에 바로 활용
- `show()` 전에 `savefig()` 호출

Python example

```
plt.bar(names, scores)
plt.savefig("chart.png") # 이미지 파일로 저장
plt.show()
```

TAKEAWAY

`savefig("파일.png")` 로 이미지 저장

이론

pandas + matplotlib 연동

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- pandas로 분석한 데이터를 바로 그래프로
- 분석 → 시각화의 자연스러운 흐름

Python example

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({"이름": ["A", "B", "C"],
                  "점수": [90, 70, 85]})
plt.bar(df["이름"], df["점수"])
plt.show()
```

TAKEAWAY

pandas로 분석한 데이터를 바로 그래프로

이론

시각화 활용 사례

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 성적 분포, 월별 매출, 인구 통계
- 설문 결과(파이), 주가 변화(선)

적용 질문

- 날씨·기온 추이
- 미니 프로젝트의 결과 대시보드!

TAKEAWAY

성적 분포, 월별 매출, 인구 통계

정리

matplotlib 정리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

오늘의 결과물

- plt.bar/plot/pie 로 그리고 plt.show() 로 띄우기
- title/xlabel/ylabel 로 설명 붙이기
- 한글은 폰트 설정 필수
- savefig 로 이미지 저장

다음 연결

- pandas 데이터를 바로 시각화

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.

실습

성적 막대그래프

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import matplotlib.pyplot as plt
plt.rcParams["font.family"] = "Malgun Gothic"
# Mac: AppleGothic
names = ["홍길동", "김철수", "이영희", "박민수"]
scores = [90, 85, 95, 70]
plt.bar(names, scores, color="skyblue")
plt.title("학생별 점수")
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

월별 매출 선그래프

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import matplotlib.pyplot as plt
plt.rcParams["font.family"] = "Malgun Gothic"
months = ["1월", "2월", "3월", "4월", "5월"]
sales = [100, 120, 90, 150, 200]
plt.plot(months, sales, marker="o",
         color="green")
plt.title("월별 매출")
# ... 계속
```

실습 포인트

- marker="o" 로 데이터 점 표시

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

그래프 그리기 (택1 이상)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

따라 하기

- 메뉴 선호도 파이 차트 (설문 결과 비율)
- 과목별 평균 막대그래프 (색 강조)

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- 일주일 기온 선그래프
- 3교시 pandas 데이터를 그래프로 시각화

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 도구로 넘어갈 준비

10 MIN BREAK

다음 교시: requests·openpyxl — 웹 데이터 가져오기 & 엑셀 자동화 + 프로젝트 안내!

작성 파일 저장

설치 상태 확인

실행 결과 캡처

프로젝트 메모

TAKEAWAY

라이브러리 실습 전후에는 설치 상태와 생성된 파일을 함께 확인합니다.

5교시

5교시 — requests·openpyxl & 미...

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 웹 API·엑셀 자동화를 체험하고, 내일 미니 프로젝트를 준비한다.

이론 20분

실습 30분

휴식 10분

이론

웹에서 데이터 가져오기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 날씨, 환율, 뉴스... 인터넷엔 실시간 데이터가 가득
- 이걸 코드로 가져올 수 있다면? → requests 라이브러리

적용 질문

- 비유: 코드가 웹사이트에 요청을 보내고 응답을 받음

TAKEAWAY

날씨, 환율, 뉴스... 인터넷엔 실시간 데이터가 가득

이론

API란?

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- API = 프로그램끼리 데이터를 주고받는 약속된 창구
- 비유: 식당의 메뉴판 — 정해진 방식으로 주문(요청)하면 음식(데이터)이 나옴

적용 질문

- 날씨 API에 "서울 날씨?" 요청 → 날씨 데이터 응답
- 대부분 JSON 형식으로 응답 (어제 배운 것!)

TAKEAWAY

API = 프로그램끼리 데이터를 주고받는 약속된 창구

이론

requests.get

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- requests.get(주소) 로 데이터 요청
- status_code 200 = 성공, 404 = 없음
- 가장 기본적인 웹 요청

Python example

```
import requests

response = requests.get("https://api.github.co
m")
print(response.status_code) # 200 = 성공
print(response.text[:100]) # 응답 내용 일부
```

TAKEAWAY

requests.get(주소) 로 데이터 요청

이론

JSON 응답 다루기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- .json() 으로 응답을 딕셔너리로 변환
- 어제 배운 딕셔너리 접근 그대로!

Python example

```
import requests

url = "https://api.exchangerate-api.com/v4/latest/USD"
response = requests.get(url)
data = response.json()    # JSON → 딕셔너리!

print(data["rates"]["KRW"]) # 원화 환율
```

TAKEAWAY

.json() 으로 응답을 딕셔너리로 변환

이론

공공 API 활용 예

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 날씨 API: 도시 → 기온·강수
- 환율 API: 통화 → 실시간 환율
- 교통 API: 버스·지하철 도착 정보

적용 질문

- 뉴스, 영화, 공공데이터포털(data.go.kr)
- 대부분 무료, 가입 후 키 발급

TAKEAWAY

날씨 API: 도시 → 기온·강수

이론

requests 활용 흐름

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 요청 → 변환 → 추출 → 활용
- 받은 데이터를 pandas·matplotlib로 분석·시각화...

Python example

```
import requests
data = requests.get(url).json() # ① 요청 +
    JSON 변환
value = data["원하는키"]        # ② 필요한 값 추출
print(value)                    # ③ 활용
```

TAKEAWAY

요청 → 변환 → 추출 → 활용

이론

BeautifulSoup 맛보기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- API가 없는 웹페이지에서 직접 데이터 추출(크롤링)
- 깊게는 x, "이런 것도 가능" 정도

Python example

```
from bs4 import BeautifulSoup
import requests

html = requests.get("https://example.com").text
soup = BeautifulSoup(html, "html.parser")
print(soup.title.text)    # 페이지 제목
```

TAKEAWAY

API가 없는 웹페이지에서 직접 데이터 추출(크롤링)

이론

openpyxl — 엑셀 자동화

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- openpyxl = 파이썬으로 엑셀 파일(.xlsx)을 만들고 다루기
- 반복되는 엑셀 작업을 자동화!

적용 질문

- 비유: 매일 하던 엑셀 정리를 코드가 1초에
- 비전공자에게 가장 실용적인 자동화

TAKEAWAY

openpyxl = 파이썬으로 엑셀 파일(.xlsx)을 만들고 다루기

이론

openpyxl — 엑셀 쓰기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- 셀에 값 넣고 save() 로 엑셀 파일 생성!

Python example

```
import openpyxl

wb = openpyxl.Workbook()    # 새 엑셀 파일
ws = wb.active              # 현재 시트
ws["A1"] = "이름"
ws["B1"] = "점수"
ws.append(["홍길동", 90])   # 한 행 추가
wb.save("result.xlsx")     # 저장
```

TAKEAWAY

셀에 값 넣고 save() 로 엑셀 파일 생성!

이론

openpyxl — 엑셀 읽기

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

개념 포인트

- load_workbook 으로 기존 엑셀 읽기
- 행마다 데이터 가져와 처리

Python example

```
import openpyxl

wb = openpyxl.load_workbook("result.xlsx")
ws = wb.active
for row in ws.iter_rows(values_only=True):
    print(row)          # 각 행을 튜플로
```

TAKEAWAY

load_workbook 으로 기존 엑셀 읽기

이론

엑셀 자동화 사례

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

핵심 개념

- 여러 csv를 합쳐 하나의 엑셀 리포트
- 매일 데이터를 받아 자동 정리·저장

적용 질문

- 셀 색·서식·차트까지 자동 적용
- 반복 업무를 클릭 한 번으로

TAKEAWAY

여러 csv를 합쳐 하나의 엑셀 리포트

정리

오늘 배운 라이브러리 총정리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

라이브러리	한 일	설치
pandas	데이터 분석	pip install pandas
matplotlib	그래프	pip install matplotlib
requests	웹 API	pip install requests
openpyxl	엑셀 자동화	pip install openpyxl

제출/마무리

- 분석(pandas)→시각화(matplotlib)→저장(o...)

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.

실습

환율 정보 가져오기

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import requests
url = "https://api.exchangerate-api.com/v4/latest/USD"
try:
    data = requests.get(url).json()
    krw = data["rates"]["KRW"]
    print(f"1달러 = {krw:,.0f}원")
# ... 계속
```

실습 포인트

- 어제 배운 예외 처리로 네트워크 오류 대비!

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

엑셀 리포트 자동 생성

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import openpyxl
students = [("홍길동", 90), ("김철수", 85), ("이영희",
95)]
wb = openpyxl.Workbook()
ws = wb.active
ws.append(["이름", "점수"])    # 헤더
for name, score in students:
    # ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

실습

스스로 실습 & 프로젝트 설계 (택1 이상)

실행해 보고, 값을 바꾸고, 결과물을 확인하는 실습

1. 따라 치기

따라 하기

- 환율 API로 여러 통화 환율 한 번에 출력
- pandas 데이터를 엑셀로 저장(openpyxl)

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- 공공데이터 API 하나 골라 데이터 가져오기
- 내일 프로젝트 주제 확정 + 기능 명세·클래스 설계 스케치

TAKEAWAY

상속과 라이브러리는 직접 실행 결과를 눈으로 확인할 때 빠르게 익숙해집니다.

정리

오늘 정리 & 마무리

객체지향 심화와 실전 라이브러리를 바로 코드로 확인

오늘의 결과물

- 상속(super) · 오버라이딩 · 다형성 · 캡슐화 · 매직 메서드
- pandas(데이터 분석) · matplotlib(시각화)
- requests(웹 API) · openpyxl(엑셀 자동화)
- 오늘 익힌 것:

다음 연결

- 7일간의 학습이 끝났습니다! 내일은 미니 프로젝트로 전부 합칩니다.

TAKEAWAY

상속과 실전 도구까지 익히면 내일 미니 프로젝트를 설계할 준비가 끝납니다.